

Guía estudio unidad 1

Regresión lineal · polinomial · robusta · redes neuronales

📖 **Cómo leer esta guía** Cada tema sigue el mismo orden: **idea en una frase** → **fórmula** → **qué significa cada símbolo** → **ejemplo con números** → **código** → **trampas de examen**. Si solo tienes 20 minutos, lee las cajas "En una frase" y "Trampa" de cada sección.

El hilo de toda la unidad (la maestra lo repite):

Modelo	¿Qué cambia?	Frase para el examen
Lineal	— (base)	Combina las variables originales con pesos β
Polinomial	Las columnas (agrega $x^2, x^3 \dots$)	Cambia la forma; sigue siendo lineal en β
Robusta (Huber/Tukey)	La pérdida	Mismo modelo, otra forma de castigar errores
Red neuronal	El aproximador	Aprende sus propias transformaciones

💡 **Frase de oro** "Huber cambia la **pérdida**; la red cambia el **aproximador**; el polinomio cambia las **columnas**."

PARTE 1 · Regresión lineal

📌 **En una frase** Buscamos la recta (o hiperplano) que deja los **errores al cuadrado** lo más pequeños posible.

1.1 El modelo simple

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

Símbolo	Qué es	En streaming
y_i	Valor real del usuario i	Minutos que vio la semana siguiente
x_i	Variable de entrada	Sesiones de la semana previa
β_0	Intercepto: valor de y cuando $x = 0$	Minutos "base"
β_1	Pendiente: cuánto cambia y si x sube 1	Minutos extra por cada sesión
ε_i	Error: lo que el modelo no explica	Gustos, ruido, azar
$\hat{y}_i$	Predicción: $\hat{\beta}_0 + \hat{\beta}_1 x_i$	Lo que el modelo cree que verá
$e_i = y_i - \hat{y}_i$	Residuo (error observado)	Si es positivo, el modelo subestimó

El gorrito $\hat{}$ significa **estimado a partir de los datos**. β es el valor verdadero (desconocido); $\hat{\beta}$ es lo que calculamos.

1.2 La pérdida OLS (mínimos cuadrados ordinarios)

$$L(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum e_i^2 \quad (\text{SSE})$$

¿Por qué al cuadrado?

1. Evita que errores positivos y negativos se cancelen.
2. Castiga **mucho más** los errores grandes: un error de 10 cuesta 100, uno de 100 cuesta 10 000.
3. La derivada es sencilla, así que existe una fórmula cerrada.

⚠ El punto 2 es a la vez su virtud y su debilidad: un solo atípico puede dominar la suma. Esto motiva la [regresión robusta](#).

1.3 Fórmulas de los coeficientes (caso simple)

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}} = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Cómo entenderlas:

- **Numerador S_{xy} :** ¿ x e y se mueven juntos? Si cuando x está arriba de su media y también, los productos son positivos y la pendiente es positiva.
- **Denominador S_{xx} :** cuánto varía x . Divide para expresar "minutos **por sesión**".
- **Intercepto:** obliga a la recta a pasar por el punto promedio $(\bar{x}, \bar{y})$.

≡ Ejemplo resuelto a mano (datos de la diapositiva "Recta vs curva")

x	y	$x - \bar{x}$	$y - \bar{y}$	producto	$(x - \bar{x})^2$
1	70	-2	-190	380	4
2	135	-1	-125	125	1
3	230	0	-30	0	0
4	355	1	95	95	1
5	510	2	250	500	4
Σ				1100	10

$$\bar{x} = 3, \bar{y} = 260 \rightarrow \hat{\beta}_1 = 1100/10 = \mathbf{110 \text{ min/sesión}} \rightarrow \hat{\beta}_0 = 260 - 110(3) = \mathbf{-70}$$

Recta: $\hat{y} = -70 + 110x \rightarrow$ predicciones 40, 150, 260, 370, 480 **Residuos:** +30, -15, -30, -15, +30. Forman una U, lo que indica que **falta curvatura** (se retoma en la Parte 2).

$$SSE = 900 + 225 + 900 + 225 + 900 = 3150 \cdot SST = 123\,250 \cdot$$

$$R^2 = 1 - 3150/123250 = \$ * 0.974 * \$ \cdot RMSE = \sqrt{3150/5} \approx \mathbf{25.1 \text{ min}}$$

Lección: **R^2 de 0.97 y aun así el modelo está mal especificado.** Los residuos lo delatan.

1.4 Forma matricial (regresión múltiple)

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon \quad \hat{\mathbf{y}} = \mathbf{X}\hat{\beta}$$

$$\underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}}_{n \times 1} = \underbrace{\begin{bmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & & & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{bmatrix}}_{n \times (p+1)} \underbrace{\begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}}_{(p+1) \times 1}$$

- Cada fila es un usuario; cada columna, una variable.
- La columna de unos existe para que β_0 se multiplique por 1: así es como entra el intercepto.
- Regla de dimensiones: $(n \times (p + 1)) \cdot ((p + 1) \times 1) = (n \times 1)$. Una predicción por usuario.
- β depende de p (número de variables), nunca de n (número de usuarios).

Solución (ecuaciones normales):

$$\hat{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

De dónde sale (por si lo piden):

1. $L(\beta) = (\mathbf{y} - X\beta)^T (\mathbf{y} - X\beta)$
2. Derivar e igualar a cero: $\nabla L = -2X^T (\mathbf{y} - X\beta) = 0$
3. $X^T X\beta = X^T \mathbf{y} \rightarrow$ despejar β .

Interpretación de $X^T (\mathbf{y} - X\hat{\beta}) = 0$: el residuo queda **perpendicular** a todas las columnas de X . Ya no queda nada que las variables puedan explicar linealmente.

⚠ ¿Cuándo falla? Si $X^T X$ **no es invertible**: una columna es combinación de otras (multicolinealidad perfecta) o hay más variables que filas ($p + 1 > n$). En la práctica el software usa QR/SVD (`np.linalg.lstsq`), que es más estable.

Interpretación de β_j en múltiple: cambio esperado en y cuando x_j sube 1 unidad, manteniendo las demás constantes.

1.5 Métricas

Métrica	Fórmula	Unidades	Para qué
MAE	$\frac{1}{n} \sum e_i $	minutos	Error típico, poco sensible a extremos
MSE	$\frac{1}{n} \sum e_i^2$	minutos ²	Lo que minimiza OLS; difícil de interpretar
RMSE	$\sqrt{MSE}$	minutos	Error típico que castiga los grandes
R ²	$1 - \frac{SSE}{SST}$	sin unidades	Qué tanto mejor que predecir la media

$$SSE = \sum (y_i - \hat{y}_i)^2 \text{ (error del modelo)} \quad SST = \sum (y_i - \bar{y})^2 \text{ (error de predecir siempre la media)}$$

🔗 **Cómo leer R²** $R^2 = 0.75 \rightarrow$ el modelo tiene **25 % del error cuadrático** que tendría predecir la media, es decir, lo reduce un 75 %. **NO** significa "acierta al 75 % de los usuarios".

- $R^2 = 0$: igual que la media. $R^2 < 0$: **peor** que la media (posible en validación).
- R^2 **no prueba causalidad** y **no** se compara entre datasets con distinta varianza (cambia SST).
- Reporta siempre el **RMSE en minutos** junto con R².

1.6 Código

python

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)    # aprende media/desv
SOLO de train
X_test_s = scaler.transform(X_test)          # aplica las de train
(sin fuga)
```

```

modelo = LinearRegression().fit(X_train_s, y_train) # agrega
intercepto solo
y_pred = modelo.predict(X_test_s) # mismo
preprocesamiento!
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)
modelo.coef_, modelo.intercept_

```

python

```

# Con statsmodels (el que usa la actividad)
import statsmodels.api as sm
A = sm.add_constant(X) # ⚠ statsmodels NO agrega la
columna de 1s solo
m = sm.OLS(y, A).fit()
m.params, m.resid, m.summary()

```

python

```

# A mano con NumPy
A = np.column_stack([np.ones(len(x)), x])
beta = np.linalg.inv(A.T @ A) @ A.T @ y # fórmula de libro
beta = np.linalg.lstsq(A, y, rcond=None)[0] # forma estable

```

⚠ **Trampa del notebook** `Regresión_Lineal.ipynb` Se entrena con `modelo.fit(X_train, ...)` **sin escalar** y se predice con `user_nuevo_scaled` **escalado**, lo que da 173 min **✗**. Lo consistente da **≈ 889.5 min**. El aviso `X does not have valid feature names` es la pista. Además es **extrapolación**: 9 sesiones (el máximo en train es 8) y 790 000 minutos previos (el máximo es ≈161 000).

🔗 ¿Escalar cambia la regresión lineal? **No cambia las predicciones ni R^2** , solo los coeficientes. Sin escalar, β = minutos por **unidad**. Escalado, β = minutos por **desviación estándar** (sirve para comparar importancia). En redes, RBF y regularización **sí importa** escalar.

PARTE 2 · Regresión polinomial (sustitución no lineal)

📌 En una frase Creamos columnas nuevas ($x^2, x^3, \log x, x_1x_2$) y seguimos usando OLS, porque el modelo sigue siendo **lineal en β** .

2.1 Fórmula

$$\hat{y} = \beta_0 + \beta_1x + \beta_2x^2 + \cdots + \beta_dx^d$$

¿Por qué se llama "lineal" si tiene x^2 ? Linealidad se refiere a los **parámetros**.

Ninguna β está elevada, multiplicada por otra β ni dentro de una función. Si llamas $x_2 := x^2$, queda $\beta_0 + \beta_1x_1 + \beta_2x_2$, que es una regresión múltiple normal.

Contraejemplo (no lineal en β): $y = \beta_0e^{\beta_1x}$ o $y = \frac{\beta_0x}{\beta_1+x}$. Esos no se resuelven con OLS directo.

Matriz de diseño $\Phi(X)$:

$$\Phi = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & & \vdots \end{bmatrix} \quad \hat{\beta} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

Número de parámetros: 1 variable de grado d tiene $d + 1$ parámetros. Si Φ tiene m columnas: m parámetros (o $m + 1$ si el intercepto va aparte).

2.2 Cómo interpretar los coeficientes

En $\hat{y} = \beta_0 + \beta_1x + \beta_2x^2$ el efecto de una sesión extra **ya no es constante**:

$$\frac{d\hat{y}}{dx} = \beta_1 + 2\beta_2x$$

- $\beta_2 > 0$: curva hacia arriba (**convexa**); cada sesión extra suma **más** que la anterior.
- $\beta_2 < 0$: curva hacia abajo (rendimientos decrecientes).

≡ Ejemplo de la diapositiva: $\hat{y} = 20 + 50x + 15(x - 1)^2$

x	Recta $20 + 50x$	$15(x - 1)^2$	Curva	Incremento
1	70	0	70	—
2	120	15	135	+65

x	Recta $20 + 50x$	$15(x - 1)^2$	Curva	Incremento
3	170	60	230	+95
4	220	135	355	+125
5	270	240	510	+155

En la recta cada sesión suma +50 siempre. En la curva los incrementos **crecen 30 en 30** (segunda diferencia constante = $2 \cdot 15$). Esa aceleración es la curvatura. Si expandes: $15x^2 + 20x + 35$. Con los datos de 1.3, el cuadrático da **residuos 0** y $R^2 = 1$; la U de residuos desaparece.

2.3 Resultados del curso (validación, 24 usuarios)

Escenario	Modelo	Parámetros	RMSE val	R^2 val	Conclusión
Original	Recta	2	84.62	0.727	—
Original	Cuadrático	3	82.02	0.743	Mejora 3 %: no hay curvatura real , así que gana la recta por simple
Curvo	Recta	2	120.80	0.871	—
Curvo	Cuadrático	3	82.02	0.940	Mejora 32 %: la curvatura es real
Curvo	Múltiple (4 var)	5	106.52	0.899	Más variables ayudan menos que la forma
Curvo	Múltiple + z_2^2	6	79.26	0.944	Solo 3 min mejor que el cuadrático 1D con el doble de parámetros

🔗 "¿Faltan variables o falta curvatura?"

- x^2 **no agrega información nueva**: es la misma variable (sesiones) con otra forma.
- Agregar variables (regresión múltiple) **no crea curvatura** automáticamente.
- Aquí el problema era la **forma**, no la falta de variables.

② ¿Por qué el cuadrático tiene RMSE ≈ 82 en ambos escenarios pero R^2 0.743 vs 0.940? Porque cambia SST. El escenario curvo tiene mucha más variación total, así que el mismo error de 82 min es proporcionalmente menor. **No compares R^2 entre respuestas distintas.**

2.4 Sobreajuste y grado

Grado	Comportamiento	Riesgo
1	Simple, interpretable	Subajuste (residuos con patrón)
2–3	Curvatura moderada	Adecuado si hay evidencia
Alto (10+)	Pasa por casi todos los puntos de train	Sobreajuste: RMSE train ↓, RMSE val ↑; extrapola fatal

Se elige con **validación**, residuos y sentido del problema; **nunca** con el R^2 de train, que siempre sube al agregar columnas.

2.5 Código

python

```
def diseno(valores, grado=1):          # de la actividad
    valores = np.asarray(valores, float).reshape(-1)
    return np.column_stack([valores**j for j in range(grado + 1)])
# [1, x, x^2, ...]

cuadratico = sm.OLS(y_train, diseno(x_train, 2)).fit()
pred_val = cuadratico.predict(disen(x_val, 2))      # mismas
columnas en validación
```

python

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
poly = make_pipeline(PolynomialFeatures(degree=2),
LinearRegression()).fit(X_train, y_train)
```

python

```
# Múltiple + curvatura (actividad): columnas [1, z1, z2, z3, z4, z2²] → 6 coeficientes
A = np.column_stack([np.ones(len(z)), z, z[:, 1]**2]) # z[:,1] = sesiones estandarizadas
```

PARTE 3 · Regresión robusta

🕒 En una frase Mismo modelo, mismas columnas, pero una pérdida que no deja que unos pocos errores enormes manden.

3.1 Problema: el cuadrado exagera los atípicos

Un error de 1200 min en OLS aporta $1200^2 = 1\,440\,000$, lo mismo que **1600 usuarios con error de 30 min**. Por eso un solo punto puede "jalar" toda la recta.

3.2 Atípico, leverage e influencia (NO son excluyentes)

Concepto	Pregunta	Se mide con	Fórmula / idea
Atípico en y	¿Su y es rara dadas sus X ?	Residuo studentizado	$e_i / (s\sqrt{1 - h_{ii}})$
Leverage	¿Sus X están lejos del resto?	h_{ii}	Diagonal de $H = X(X^T X)^{-1} X^T$
Influencia	¿Si lo quito cambia el modelo?	Distancia de Cook	Combina residuo y leverage

Matriz sombrero H : $\hat{y} = Hy$ ("le pone el gorrito a y").

- $h_{ii} \in [0, 1]$ indica cuánto **pesa y_i en su propia predicción**.
- $\sum h_{ii} = q$ (número de columnas), así que el promedio es q/n .
- Regla: revisar si $h_{ii} > 2q/n$.

Distancia de Cook:

$$D_i = \frac{e_i^2}{q \cdot MSE} \cdot \frac{h_{ii}}{(1 - h_{ii})^2}$$

Es **residuo grande** × **leverage alto**. Si falta uno de los dos, Cook es pequeño. Regla: revisar si $D_i > 4/n$.

≡ Los casos A, L, I (base: 28 meses, 3 sesiones, calificación 2.6)

Caso	Historial	Observado – predicción	h	Cook (solo)	Tipo
A	27 214 (normal)	+ 1200	0.024	0.24	Atípico en y, X normal
L	1 000 000 (extremo)	0	0.981	0.000	Leverage alto, no influyente
I	1 000 000 (extremo)	+ 1200	0.981	65.8	Leverage + atípico = muy influyente

Lección 1: *leverage no implica influencia*. L está lejísimos en X, pero cae justo sobre la recta. **Lección 2:** si se agregan **juntos**, L e I comparten la misma X extrema. OLS "parte la diferencia" (residuos −595 y +605), y **L se vuelve influyente** (Cook 3.38). El diagnóstico depende del resto de los datos.

3.3 Idea general: M-estimadores

En vez de minimizar $\sum r_i^2$, minimizamos:

$$\min_{\beta} \sum_{i=1}^n \rho(r_i), \quad r_i = y_i - \hat{y}_i$$

Nombre	$\rho(r)$	Resultado
OLS	$\frac{1}{2} r^2$	Media condicional; sensible a atípicos
L1 / MAE	$ r $	Mediana condicional; robusto, pero no es suave en 0
Huber	Cuadrática cerca, lineal lejos	Lo mejor de ambos
Tukey	Crece y luego se aplana	Ignora los extremos

3.4 Pérdida de Huber (fórmula central)

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & \text{si } |r| \leq \delta \\ \delta \left(|r| - \frac{\delta}{2} \right) & \text{si } |r| > \delta \end{cases}$$

Qué significa cada parte:

- δ es el **umbral**: hasta dónde un error se considera "normal" (en minutos).
- **Zona** $|r| \leq \delta$: idéntica a OLS. Errores chicos se tratan igual.
- **Zona** $|r| > \delta$: crece **en línea recta**, no al cuadrado.
- **¿Por qué el $-\delta/2$?** Para que las dos piezas **se unan sin salto**. En $r = \delta$: la pieza cuadrática da $\frac{1}{2}\delta^2$ y la lineal da $\delta(\delta - \frac{\delta}{2}) = \frac{1}{2}\delta^2$. Coinciden, así que la función es **continua y suave**.

La derivada (función de influencia ψ): explica la frase de la maestra.

$$\psi(r) = L'_{\delta}(r) = \begin{cases} r & |r| \leq \delta \\ \delta \cdot \text{signo}(r) & |r| > \delta \end{cases} = \text{clip}(r, -\delta, \delta)$$

🌀 "Huber no acota la pérdida, acota su pendiente"

- En OLS la derivada es r : un error de 1200 **empuja** con fuerza 1200.
- En Huber la derivada se **topa en δ** : un error de 1200 empuja con fuerza 30, igual que uno de 31.
- La pérdida sigue creciendo (no está acotada), pero su **influencia** sobre β sí.

☰ Cálculo ($\delta = 30$), la tabla del examen

r	OLS $\frac{1}{2}r^2$	Huber	Operación
10	50	50	$ 10 \leq 30$, así que $\frac{1}{2}(10)^2$
30	450	450	Justo en el borde, coinciden
100	5 000	2 550	$30(100 - 15) = 30 \cdot 85$
1200	720 000	35 550	$30(1200 - 15) = 30 \cdot 1185$

En OLS el error de 1200 pesa **14 400 veces** más que el de 10. En Huber pesa **711 veces** más.

3.5 De la pérdida a los pesos (IRLS)

Huber se resuelve como **mínimos cuadrados ponderados** repetidos. El peso de cada punto es:

$$w(r) = \frac{\psi(r)}{r} = \min \left(1, \frac{\delta}{|r|} \right)$$

$$\hat{\beta} = (X^T W X)^{-1} X^T W \mathbf{y}, \quad W = \text{diag}(w_1, \dots, w_n)$$

Algoritmo IRLS (Iteratively Reweighted Least Squares):

1. Ajustar OLS y obtener residuos r_i .
2. Estimar la **escala robusta** s (tamaño típico del error, sin que los extremos la inflen): $s = \frac{\text{mediana}(|r_i - \text{mediana}(r)|)}{0.6745}$ (MAD normalizada).
3. $\delta = 1.345 s$. En el ejemplo de clase: $s \approx 40.09$, así que $\delta \approx 53.92$ min.
4. Calcular pesos w_i y reajustar con mínimos cuadrados ponderados.
5. Repetir 2–4 hasta que β deje de cambiar.

✍ **¿Por qué 1.345? Es la constante que hace que Huber tenga 95 % de la eficiencia de OLS si los errores fueran normales. Casi no pierdes nada cuando no hay atípicos y ganas mucho cuando sí los hay.**

Comprobación rápida con los pesos de la diapositiva: A tiene residuo ≈ 1200 , así que $w = 53.92/1200 \approx 0.045$ ✓. L tiene residuo ≈ 0 , así que $w = 1$ ✓.

3.6 Tukey (bisquare) y comparación de pesos

Con residuo escalado $u = r/s$ y umbral c (típico 4.685):

Técnica	Peso $w(u)$	Residuo enorme
OLS	1	Domina el ajuste
Huber	1 si $ u \leq c$; $c/ u $ si $ u > c$	Peso pequeño pero > 0

Técnica	Peso $w(u)$	Residuo enorme
Tukey	$[1 - (u/c)^2]^2$ si $ u < c$; 0 si $ u \geq c$	Peso 0: no participa

Punto	Peso Huber	Peso Tukey
A	0.045	0
L	1	1
I	0.047	0

⚠ Trampas clásicas

- **Peso 0 $\neq$ error 0.** El dato no se borra; su error **sigue contando en el RMSE**.
- **Peso bajo $\neq$ dato incorrecto.** Puede ser un usuario real que hizo maratón. Hay que investigar.
- Un peso **no es una probabilidad** de error.
- Robusta **no corrige leverage** automáticamente (L tiene peso 1) ni sustituye buscar mejores variables.
- Tukey no es convexa: puede depender del punto de inicio. Por eso se arranca con Huber u OLS.

3.7 Resultados de la actividad (75 train con A, L, I; 24 validación)

Modelo	RMSE train	RMSE val	MAE val	R ² val
OLS múltiple + curva	176.7	84.9	59.0	0.936
Huber $\delta = 30$	200.8	81.9	52.3	0.941

- Huber tiene **peor train y mejor validación**: no persigue a A e I, que están en train; validación no tiene atípicos.
- Mismas 6 columnas en ambos, así que la mejora se debe **solo a la pérdida**.

Barrido de δ (qué pasa al moverlo):

δ	RMSE val	Pesos A/I	Comportamiento
5	80.7	≈ 0.004	Casi L1/mediana
30	81.9	≈ 0.025	Robusto
150	79.9 (mejor)	≈ 0.13	Robusto
1200	84.9	1.0	= OLS (todos los residuos $< \delta$)

✓ **Conclusión de la actividad $\delta = 30$ es una buena elección.** El mejor candidato (150) gana solo ≈ 2 min; la curva es casi plana entre 5 y 300. Límites: $\delta \rightarrow 0$ da L1; $\delta \rightarrow \infty$ da OLS. El test **no se abre** para elegir δ .

3.8 Código

python

```
# Huber con  $\delta$  FIJO en minutos (la actividad)
from scipy.optimize import least_squares
beta0 = np.linalg.lstsq(A, y, rcond=None)[0] #
arranque OLS
fit = least_squares(lambda b: A @ b - y, beta0, loss='huber',
f_scale=30)
beta_huber = fit.x
pesos = np.minimum(1, 30 / np.abs(y - A @ beta_huber)) # w =
min(1,  $\delta/|r|$ )

def perdida_huber(r, delta):
    return np.where(abs(r) <= delta, .5*r*r, delta*(abs(r) -
.5*delta))
```

python

```
# Huber / Tukey con escala estimada (IRLS)
import statsmodels.api as sm
huber = sm.RLM(y, A, M=sm.robust.norms.HuberT(t=1.345)).fit()
tukey = sm.RLM(y, A, M=sm.robust.norms.TukeyBiweight(c=4.685)).fit()
huber.weights # pesos finales de cada registro
```

```
from sklearn.linear_model import HuberRegressor # epsilon=1.35
actúa sobre r/s
# ⚠ HuberRegressor(epsilon=30) NO equivale a  $\delta = 30$  minutos
```

python

```
# Diagnóstico
inf = sm.OLS(y, A).fit().get_influence()
inf.hat_matrix_diag # leverage h
inf.cooks_distance[0] # Cook
inf.resid_studentized_internal # residuo studentizado
```

PARTE 4 · Redes neuronales

📌 En una frase En vez de diseñar nosotros las columnas ($x^2 \dots$), la red **aprende sus propias transformaciones con pesos y activaciones no lineales.**

4.1 Arquitectura: una capa oculta

$$H = \sigma(XW_1 + \mathbf{1}b_1^T) \quad \hat{y} = HW_2 + \mathbf{1}b_2$$

Símbolo	Dimensión	Qué es
X	$n \times p$	Datos: n usuarios, p variables
W_1	$p \times h$	Pesos entrada $\rightarrow$ oculta (cada columna es una neurona)
b_1	h	Un sesgo por neurona oculta
σ	—	Activación no lineal (ReLU, tanh, sigmoide)
H	$n \times h$	Activaciones ocultas: las nuevas columnas aprendidas
W_2	$h \times 1$	Pesos oculta $\rightarrow$ salida
b_2	1	Sesgo de salida
$\hat{y}$	$n \times 1$	Una predicción por usuario
θ	—	Conjunto de todos los W y b (la red no tiene un solo β)

Verificar dimensiones: $(n \times p)(p \times h) = n \times h \rightarrow \sigma \rightarrow (n \times h)(h \times 1) = n \times 1 \checkmark$

Una neurona, por dentro: $a_j = \sigma(w_{1j}x_1 + w_{2j}x_2 + \dots + w_{pj}x_p + b_j)$. Es una regresión lineal **pasada por una función no lineal**.

Parámetros:

$$\# \theta = \underbrace{p \cdot h}_{W_1} + \underbrace{h}_{b_1} + \underbrace{h}_{W_2} + \underbrace{1}_{b_2} = ph + 2h + 1$$

≡ **Actividad del curso:** $p = 4, h = 3$ $W_1 : 4 \times 3 = 12 \cdot b_1 : 3 \cdot W_2 : 3 \times 1 = 3 \cdot b_2 : 1 \rightarrow$ **19 parámetros. ¿Es adecuada?** Es válida, pero con 120 registros quedan unos 72 para train, es decir, unas 4 filas por parámetro. Hay que compararla contra la media, la recta y el polinomio (mismas particiones), escalar, regularizar o usar parada temprana, y probar varias semillas.

Para más capas: suma (entradas $\times$ salidas + salidas) por cada capa. Ej.

$$4 \rightarrow 8 \rightarrow 4 \rightarrow 1: (4 \cdot 8 + 8) + (8 \cdot 4 + 4) + (4 \cdot 1 + 1) = 40 + 36 + 5 = 81.$$

4.2 ¿Por qué hace falta la activación?

Sin activación ($\sigma(z) = z$):

$$\hat{y} = (XW_1 + b_1)W_2 + b_2 = \underbrace{X(W_1W_2)}_{\text{un vector}} + \underbrace{(b_1W_2 + b_2)}_{\text{un número}}$$

Es exactamente **una regresión lineal**. Apilar capas lineales sigue dando un **modelo afín**. La no linealidad es lo que permite curvas.

Activación	Fórmula	Nota
ReLU	$\max(0, z)$	La más usada; derivada 0 o 1
Sigmoide	$1/(1 + e^{-z})$	Salida entre 0 y 1
tanh	$\tanh z$	Entre -1 y 1

En la **salida** de regresión no se pone activación (queremos cualquier número real de minutos).

≡ **Forward pass a mano (1 entrada, 2 neuronas ReLU)** $W_1 = [1, -1]$, $b_1 = [0, 1]$, $W_2 = [2, 3]^T$, $b_2 = 1$. **Usuario con $x = 2$ (escalado):**

1. $z = xW_1 + b_1 = [2 \cdot 1 + 0, 2 \cdot (-1) + 1] = [2, -1]$
2. $a = \text{ReLU}(z) = [2, 0]$ (la neurona 2 se "apaga")
3. $\hat{y} = 2 \cdot 2 + 0 \cdot 3 + 1 = 5$

4.3 Cómo aprende: pérdida, backpropagation y gradiente

1. **Forward:** calcular $\hat{y}$ y la pérdida, por ejemplo $L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$. También puede ser **Huber**: la pérdida es otra decisión independiente.
2. **Backpropagation:** calcular $\nabla_{\theta} L$ con la **regla de la cadena**, de la salida hacia atrás.
3. **Actualizar** (descenso de gradiente):

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

- η = **tasa de aprendizaje** (tamaño del paso). Muy grande, diverge; muy chica, es lentísimo.
 - El signo menos significa bajar la pendiente de la pérdida.
4. Repetir por **épocas** (pasadas completas por train), vigilando la pérdida de train y de validación.

≡ **Un paso de gradiente (continuando el ejemplo, y real = 7) Pérdida**

$$L = \frac{1}{2} (\hat{y} - y)^2 = \frac{1}{2} (5 - 7)^2 = 2 \rightarrow \frac{\partial L}{\partial \hat{y}} = \hat{y} - y = -2$$

Capa de salida (regla de la cadena, $\hat{y} = a \cdot W_2 + b_2$):

- $\frac{\partial L}{\partial b_2} = -2 \cdot 1 = -2$
- $\frac{\partial L}{\partial W_2} = -2 \cdot a = -2 \cdot [2, 0] = [-4, 0]$

Capa oculta (retroceder un paso más):

- Neurona 1: $\frac{\partial L}{\partial w_1} = \underbrace{-2}_{\partial L / \partial \hat{y}} \cdot \underbrace{2}_{W_2} \cdot \underbrace{1}_{\text{ReLU}'(2)} \cdot \underbrace{2}_x = -8$
- Neurona 2: $\text{ReLU}'(-1) = 0$, así que el gradiente es **0**: no aprende con este dato (neurona apagada).

Actualizar con $\eta = 0.1$: $b_2 = 1 - 0.1(-2) = 1.2$; $W_2 = [2.4, 3]$; $w_1 = 1 - 0.1(-8) = 1.8$. Nuevo forward: $z_1 = 3.6 \rightarrow \hat{y} = 3.6 \cdot 2.4 + 1.2 = 9.84$. Se pasó de 7: con η grande se "brinca" el mínimo. Por eso importa la tasa de aprendizaje.

4.4 Teorema de aproximación universal

$$\forall x \in K : |f(x) - \hat{f}(x)| < \varepsilon$$

Léelo así: "para cualquier función continua f y cualquier tolerancia $\varepsilon > 0$, **existe** una red de **una capa oculta** (con suficientes neuronas, sesgos y activación continua **no polinómica**, como ReLU) cuyo error es menor que ε en **todo** el dominio K ."

Símbolo	Significa
$\forall$	"para todo"
K	Dominio cerrado y acotado (incluye extremos, límites finitos). Ej. $K = [1, 14]$ sesiones
$\hat{f}$	La función que calcula la red
ε	Tolerancia de error que tú eliges

🔗 Las tres cosas que la maestra distingue

	¿Lo garantiza el teorema?
Capacidad: existe una red que aproxima f	✅ Sí
Optimización: el entrenamiento la encuentra	❌ No
Generalización: funciona con datos nuevos y pocos datos	❌ No
Fuera de K (extrapolación, ej. 20 sesiones)	❌ No

"Una red más grande no garantiza resolver las tres."

¿Por qué "no polinómica"? Si σ fuera un polinomio de grado k , la red completa sería un polinomio de grado $\leq k$ y no podría aproximar cualquier función.

4.5 Buenas prácticas (lista de la diapositiva)

- **Preparar:** imputar y escalar **solo con train** (las redes son muy sensibles a la escala).
- **Comparar:** mismas particiones y métricas que OLS y el polinomio.
- **Controlar:** regularización, **parada temprana** (detener cuando la pérdida de validación deja de bajar) y **varias semillas** (la inicialización es aleatoria).
- **Explicar:** residuos, sensibilidad, resultados por tipo de usuario.

Arquitecturas según la estructura de los datos: MLP para tablas ·

RNN/LSTM/atención para secuencias (orden de sesiones) · conexiones residuales para redes profundas. *"Más profundidad no sustituye más evidencia ni mejores variables."*

4.6 Redes de funciones de base radial (RBF)

$$\phi_j(x) = \exp\left(-\frac{\|x - c_j\|^2}{2\sigma_j^2}\right) \quad \hat{y} = \beta_0 + \sum_{j=1}^m \beta_j \phi_j(x)$$

Símbolo	Qué es
c_j	Centro: dónde responde la neurona
σ_j	Ancho: qué tan lejos responde
$\ x - c_j\ $	
ϕ_j	Vale 1 en el centro y cae a 0 lejos (campana)

≡ **Centro** $c = 3$ sesiones, $\sigma = 1$ $x = 3$: $\phi = e^0 = 1$ · $x = 4$: $\phi = e^{-1/2} \approx 0.61$ ·
 $x = 6$: $\phi = e^{-9/2} \approx 0.011$

- **Ancho pequeño:** muy local, riesgo de ajustar ruido (sobreajuste).
- **Ancho grande:** muy suave, riesgo de perder detalles (subajuste).
- Usa **distancias**, así que hay que **estandarizar**. Si no, minutos_previos (≈ 10000) aplasta a sesiones (≈ 3).

Parámetros:

- Centros y anchos **fijos**: $A = [\mathbf{1}, \Phi]$ de $n \times (m + 1)$, y se ajustan $m + 1$ coeficientes **con OLS o Huber**. Es otra vez una regresión lineal en β .
- **Todo entrenado**: mp (centros) + m (anchos) + $m + 1$ (salida) = $mp + 2m + 1$.

4.7 ¿Qué aproximador conviene? (reto final)

	Polinomio	RBF	MLP
Bases	Pocas, diseñadas	Locales	Aprendidas
Ajuste	OLS, fácil de inspeccionar	OLS si centros fijos	Iterativo, delicado
Riesgo	Extrapolación de grado alto	Cobertura y escala	Sobreajuste, semillas, pocos datos

¿Qué evidencia justificaría pasar del polinomio a una red? Que la red baje el RMSE de validación **de forma consistente entre semillas** y en magnitud relevante (minutos); que los residuos del polinomio muestren estructura no capturada (interacciones); que haya **suficientes datos** para sus parámetros; y que la pérdida de interpretabilidad valga la pena. Si no, gana el polinomio.

4.8 Código (por si preguntan)

python

```
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
red = make_pipeline(
    StandardScaler(),                                # escalar es
obligatorio
    MLPRegressor(hidden_layer_sizes=(3,),            # 1 capa oculta,
h = 3
                    activation='relu', alpha=1e-3,   # alpha =
regularización L2
                    early_stopping=True,             # parada
temprana
                    learning_rate_init=0.01,         #  $\eta$ 
                    max_iter=2000, random_state=42)) # probar varias
```

```
semillas
red.fit(X_train, y_train)
```

python

```
# Forward pass en NumPy
H      = np.maximum(0, X @ W1 + b1)    # ReLU; X: n×p, W1: p×h → n×h
y_hat  = H @ W2 + b2                  # W2: h×1 → n×1
```

Resumen de fórmulas (para copiar en la hoja)

Tema	Fórmula	Recordatorio
Recta	$\hat{y} = \beta_0 + \beta_1 x$	$\beta_1 = \text{min por sesión}$
Pendiente	$\hat{\beta}_1 = S_{xy}/S_{xx}$	Covariación / variación de x
Intercepto	$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$	Pasa por $(\bar{x}, \bar{y})$
OLS matricial	$\hat{\beta} = (X^T X)^{-1} X^T y$	Requiere $X^T X$ invertible
Pérdida OLS	$\sum (y_i - \hat{y}_i)^2$	Castiga al cuadrado
R ²	$1 - SSE/SST$	% menos error que la media
RMSE	$\sqrt{\frac{1}{n} \sum e_i^2}$	En minutos
Polinomio	$\beta_0 + \beta_1 x + \beta_2 x^2$	Lineal en β , $d + 1$ parámetros
Efecto marginal	$\beta_1 + 2\beta_2 x$	Ya no es constante
Leverage	h_{ii} de $H = X(X^T X)^{-1} X^T$	Revisar si $> 2q/n$
Cook	$\frac{e_i^2}{q MSE} \frac{h_{ii}}{(1-h_{ii})^2}$	Revisar si $> 4/n$
Huber	$\frac{1}{2} r^2 / \delta(r - \frac{\delta}{2})$	Acota la pendiente
Peso Huber	$\min(1, \delta/ r)$	Nunca 0
δ robusto	$1.345 s$, con $s = \text{MAD}/0.6745$	95 % eficiencia
Peso Tukey	$[1 - (u/c)^2]^2$ o 0	Puede ser 0
Red	$\hat{y} = \sigma(XW_1 + b_1)W_2 + b_2$	$ph + 2h + 1$ parámetros
Gradiente	$\theta \leftarrow \theta - \eta \nabla L$	η = tasa de aprendizaje

Tema	Fórmula	Recordatorio
RBF	$e^{-\ x-c\ ^2/2\sigma^2}$	$m + 1$ o $mp + 2m + 1$

Autoevaluación rápida

?

1. ¿ $y = \beta_0 + \beta_1 x + \beta_2 x^2$ es lineal? **Sí, en β . Se resuelve con OLS usando las columnas $[1, x, x^2]$.**

?

2. ¿ $y = \beta_0 e^{\beta_1 x}$ es lineal en β ? **No: β_1 está dentro de la exponencial. (Si tomas log de ambos lados, $\ln y = \ln \beta_0 + \beta_1 x$ sí lo es.)**

?

3. Una red $5 \rightarrow 4 \rightarrow 1$, ¿cuántos parámetros tiene? $5 \cdot 4 + 4 + 4 \cdot 1 + 1 = 29$.

?

4. Con $\delta = 20$, ¿cuánto vale la pérdida Huber de $r = 50$? $20(50 - 10) = 800$ (OLS: 1250). Peso: $20/50 = 0.4$.

?

5. ¿Qué pasa con Huber si δ es mayor que todos los residuos? Todos los pesos valen 1, así que Huber = **OLS**. Si $\delta \rightarrow 0$, se parece a **L1 (mediana)**.

?

6. ¿Por qué la red sin activación es una regresión lineal? Porque $(XW_1 + b_1)W_2 + b_2 = X(W_1W_2) + \text{cte}$: una composición de funciones lineales es lineal.

?

7. ¿Leverage alto implica influencia? No. L tiene $h = 0.98$ y $\text{Cook} = 0$ porque cae sobre la recta. Influencia = leverage y residuo grande.

?

8. ¿Qué garantiza el teorema de aproximación universal? Solo que **existe** una red que aproxima f dentro de K . No garantiza entrenarla, ni generalizar, ni nada fuera de K .

?

9. ¿Por qué Huber tiene peor RMSE en train y mejor en validación? Train contiene A e I (+1200). Huber no los persigue (peso ≈ 0.03), así que su error ahí es grande, pero ajusta mejor a los usuarios normales, que son los de validación.

?

10. Si $R^2 = 0.97$ pero los residuos forman una U, ¿el modelo está bien? No. Falta curvatura; agrega x^2 . R^2 alto no garantiza que la forma sea correcta.

?

11. ¿Tukey borra el dato? No. Le da peso 0 en el reajuste, pero su error sigue contando en el RMSE. Hay que investigarlo, no descartarlo.

?

12. ¿Para qué sirve el $-\delta/2$ en Huber? Para que las dos piezas se unan de forma continua en $|r| = \delta$ (ambas valen $\delta^2/2$).

?

13. ¿Qué pasa si η es muy grande? Los pasos brincan el mínimo y la pérdida oscila o diverge. Si es muy chica, el entrenamiento es lentísimo.

?

14. ¿Por qué hay que escalar en RBF y redes, pero no importa en OLS? RBF usa distancias y las redes usan gradientes: variables con escala grande dominan. OLS da las mismas predicciones con o sin escalar; solo cambian los coeficientes.